

**SMART LOST & FOUND SYSTEM USING AI
TRACE FIND - CLOUD-BASED ITEM MATCHING SYSTEM**

CLOUD COMPUTING

Submitted by

HARSHINI MANICKAM (230701111)

SHALINI RK (230701304)

SENTHAMIZH SELVI V (230701301)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

NOVEMBER 2025

ANNA UNIVERSITY, CHENNAI



BONAFIDE CERTIFICATE

Certified that this project report titled "TRACE FIND - SMART LOST & FOUND SYSTEM USING AI" is the bonafide work of HARSHINI MANICKAM (230701111), SHALINI R K (230701304), SENTHAMIZH SELVI V (230701301) who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy
Professor & Supervisor
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

SIGNATURE

Dr. M. Ayyadurai
Associate Professor
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on_____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman Mr. S.MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Dr.MALATHY E.M, Ph.D., Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, DR. M. AYYADURAI M.E.,Ph.D., Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

HARSHINI MANICKAM
SHALINI RK
SENTHAMIZH SELVI V

ABSTRACT

Lost items are a persistent and disruptive problem in campuses, offices, and public places. Existing systems rely entirely on manual logs and physical notice boards, which offer no automated way to match lost items with found ones. This results in low recovery rates, poor user visibility, and significant frustration for those who have lost belongings.

Trace Find is an AI-powered, cloud-based Smart Lost & Found System designed to address these inefficiencies. The system enables users to report lost or found items by uploading an image and a textual description. Using a hybrid AI matching engine that combines image similarity and text similarity through embeddings, the system automatically finds potential matches between lost and found reports without any manual search required.

The architecture is fully serverless and event-driven, built on AWS. When a new item is reported, it is stored in Amazon DynamoDB and Amazon S3, triggering an orchestration Lambda function. This function generates combined image and text embeddings using Amazon Bedrock and the CLIP model (Hugging Face), stores them in a Vector Database, and runs a nearest-neighbour similarity search. If cosine similarity exceeds a threshold of 0.75, a match is confirmed and users are notified in real time via Amazon SES/SNS.

The platform is highly scalable, cost-efficient, and requires no manual server management. Key modules include identity and access management via Amazon Cognito, item reporting, the AI matching engine, claims and ownership verification, and an automated notification system. The expected outcome is a significant improvement in item recovery rates and a seamless, automated experience for users across institutional environments.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	Introduction	1
	1.1 Problem Statement	2
	1.2 Objective of the Project	4
	1.3 Scope and Boundaries	6
	1.4 Stakeholders and End Users	8
	1.5 Technologies Used	10
	1.6 Organization of the Report	12
2	System Design and Architecture	14
	2.1 Requirement Summary (Functional & Non-Functional)	15
	2.2 Proposed Solution Overview	17
	2.3 Cloud Deployment Strategy	19
	2.4 Infrastructure Requirements	21
	2.5 AWS Services Mapping and Justification	23
3	AI Implementation	26
	3.1 AI Matching System	27
	3.2 Embedding Generation (Image & Text)	29
	3.3 Vector Database and Similarity Search	31
	3.4 Event-Driven Architecture	33
	3.5 Notification System	35
4	Cloud Operations and Security	38
	4.1 Authentication and Access Control	39
	4.2 Monitoring and Observability	41
	4.3 Serverless Operations	43
	4.4 Disaster Recovery Planning	45
5	Results and Discussion	48
	5.1 Implementation Summary	49
	5.2 Challenges Faced and Resolutions	51
	5.3 Performance or Cost Observations	53
	5.4 Key Learnings and Team Contributions	55
6	Conclusion and Future Enhancement	58

	6.1 Conclusion	59
	6.2 Future Scope	61
	References	63
	Appendices	65

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Lost items are a persistent challenge in academic campuses, corporate offices, and public spaces. Traditional approaches to managing lost and found items are manual, fragmented, and inefficient, leading to a range of significant problems:

- **Reliance on Manual Processes:** Existing systems rely entirely on physical notice boards, manual logbooks, and word-of-mouth communication. There is no systematic way to register or track lost items, and the process depends entirely on the effort of individuals to check boards or visit lost-property offices.
- **No Automated Matching:** There is no automated mechanism to connect a person who has lost an item with someone who has found it. Even when both parties exist within the same campus or building, a match is rarely made because the information is siloed and inaccessible.
- **Low Recovery Rate:** Due to the inefficiency of manual systems, the recovery rate for lost items in institutional environments is extremely low. Most items that are found are never reunited with their original owners, leading to waste and financial loss.
- **Poor User Visibility:** Users have no easy way to search for their lost items or to report a found item to a wide audience. The absence of a digital, searchable platform means that relevant information never reaches the right person.
- **Lack of Trust and Verification:** Existing informal methods (e.g., social media groups) lack ownership verification mechanisms, making it difficult to confirm that a claimant is the true owner of a found item.

Combined, these issues result in significant material losses for individuals, frustration among campus community members, and an unrealised potential for technology to solve a very common and relatable problem.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design and implement an AI-powered, cloud-native Lost & Found system that automates the process of matching lost items with found ones, thereby significantly improving recovery rates and user experience.

1. **Build an Automated AI Matching System:** Develop a hybrid AI engine that uses both image similarity and text similarity to automatically find matches between lost and found item reports, eliminating the need for manual searching.
2. **Enable Easy Reporting:** Allow users to report lost or found items by simply uploading a photo and a brief description, making the process as frictionless as possible through a user-friendly interface.
3. **Deliver Real-Time Notifications:** Implement an event-driven notification system that instantly alerts users when a potential match for their lost item is found, ensuring timely recovery opportunities.
4. **Build a Scalable and Cost-Efficient Cloud Platform:** Leverage a fully serverless, event-driven architecture on AWS to ensure the system can scale to thousands of concurrent users without manual infrastructure management, while remaining cost-efficient.
5. **Ensure Secure Ownership Verification:** Implement a structured claims module that allows users to formally claim a found item and provides a process for verifying ownership before a handover is arranged.

1.3 SCOPE AND BOUNDARIES

The scope of this project encompasses the complete design and cloud deployment of a full-stack AI-powered application. This includes user authentication, item reporting, AI-based matching, real-time notification, and a claims workflow.

The system supports core functionalities such as user registration and authentication via Amazon Cognito, image and description uploads to Amazon S3 and DynamoDB, automatic AI embedding generation, vector similarity search, and email/SMS notifications via Amazon SES and SNS.

The AI pipeline is built on a serverless Lambda architecture, with Amazon Bedrock providing text embeddings and the CLIP model (via Hugging Face or Bedrock) providing image embeddings. The entire infrastructure is managed on AWS with a focus on a serverless, event-driven design.

The primary scope is an institutional deployment (college campus), though the architecture is designed to scale to broader environments. The system does not currently include a mobile application; it is accessible via a web interface.

1.4 STAKEHOLDERS AND END USERS

Stakeholders: Project Developers / Team Members - Design, build, and maintain platform modules such as authentication, item reporting, AI matching, embedding generation, and notification services.

End Users:

- 6. Campus Students and Staff who have lost or found items and want a quick, reliable way to report and recover them.
- 7. Security and Administrative Personnel who manage the physical handover of items once a match has been confirmed and a claim verified.
- 8. New Users seeking to quickly locate a lost item by uploading a photo for AI-based matching.

1.5 TECHNOLOGIES USED

The project incorporates a modern, serverless technology stack built on AWS, tailored for scalability, AI integration, and operational efficiency:

Table 1. AWS Services

Component	Service	Purpose
Authentication	Amazon Cognito	User login & JWT token management
API Layer	Amazon API Gateway	Handle and route user requests
Compute	AWS Lambda	Serverless backend logic & AI orchestration
Database	Amazon DynamoDB	Store item metadata & match results
Image Storage	Amazon S3 Bucket	Store uploaded item images
AI Service	Amazon Bedrock / CLIP	Generate text and image embeddings
Notifications	Amazon SES / SNS	Send real-time email and SMS alerts
Monitoring	Amazon CloudWatch	Logs, metrics, and system observability

Table 2. AI and Matching Technologies

Technology	Purpose
CLIP Model (Hugging Face / Bedrock)	Image-to-vector embedding generation
Amazon Bedrock Embeddings	Text-to-vector embedding generation
Vector Database	Store and search embedding vectors
Cosine Similarity (Nearest Neighbour)	Compute similarity between item embeddings

1.6 ORGANIZATION OF THE REPORT

This report is structured to provide a complete and logical documentation of the entire project lifecycle.

Chapter 1 introduces the problem context, project objectives, scope, stakeholders, technologies, and report structure.

Chapter 2 presents a detailed examination of the system design and architecture, including requirements, the proposed solution overview, cloud strategy, infrastructure specifications, and AWS service mappings.

Chapter 3 delves into the AI implementation, covering the hybrid matching system, embedding generation for images and text, the vector database, the event-driven architecture, and the notification pipeline.

Chapter 4 focuses on cloud operations and security, discussing authentication, access control, monitoring, serverless operations, and disaster recovery planning.

Chapter 5 presents the results of the implementation, including summaries, challenges overcome, performance metrics, and key learnings.

Chapter 6 concludes the report with a summary of achievements and a roadmap for future enhancements.

The document concludes with references and appendices containing system architecture diagrams and configuration details.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The system consists of multiple cloud-based serverless components deployed on Amazon Web Services (AWS). Each module is designed for independence, scalability, and minimal operational overhead.

User-facing interactions are handled through an API layer built on Amazon API Gateway, which routes requests to the appropriate AWS Lambda functions. Authentication is enforced via Amazon Cognito, which provides JWT-based session management and user pool management at no significant cost for low to moderate usage.

When a user reports a lost or found item, the item metadata (title, description, category, location, contact) is stored in Amazon DynamoDB, and the uploaded image is saved to an Amazon S3 bucket. An S3 event trigger then invokes Lambda 1, the orchestration function, which initiates the AI pipeline.

The AI pipeline generates a combined embedding from the item image (using CLIP) and text description (using Amazon Bedrock). These embeddings are stored in a Vector Database and a similarity search is performed to find the nearest neighbours. If any match exceeds the cosine similarity threshold of 0.75, the match is confirmed and Lambda 2 (the notification function) is invoked to alert the relevant users via Amazon SES (email) and SNS (SMS).

Non-Functional Requirements

The system is designed to meet a comprehensive set of non-functional requirements ensuring performance, security, reliability, and cost-efficiency.

- **Performance:** The platform targets API response times below 300ms for standard CRUD operations. AI embedding and matching operations are expected to complete within 3-5 seconds, well within the tolerance for asynchronous background processing.

- **Scalability:** The serverless architecture on AWS Lambda and DynamoDB inherently supports auto-scaling to handle any number of concurrent users without manual intervention, making it suitable for campuses of any size.
- **Availability:** By leveraging AWS managed services, the system benefits from AWS SLAs guaranteeing high availability (99.9%+) for all core components.
- **Security:** Access is controlled through Amazon Cognito JWT tokens. API Gateway enforces authorisation on all protected routes. Sensitive configuration is managed through AWS Secrets Manager and environment variables.
- **Cost-Efficiency:** The pay-per-invocation model of AWS Lambda, the on-demand pricing of DynamoDB, and the free tier for SES means the platform has near-zero cost at low usage levels, making it ideal for institutional deployments with limited budgets.

2.2 PROPOSED SOLUTION OVERVIEW

Trace Find is a cloud-native, event-driven platform that separates concerns across three primary layers: the User Interface, the Backend Services layer, and the AI Pipeline.

At the presentation layer, a web application provides the user interface for reporting and searching items. Users authenticate via Amazon Cognito and interact with the system through Amazon API Gateway.

The backend services layer handles all business logic through AWS Lambda functions. DynamoDB stores structured item metadata and match records. Amazon S3 stores uploaded images. When a new item image is uploaded to S3, it triggers the core AI orchestration Lambda.

The AI Pipeline is the core differentiator of the system. Lambda 1 (Orchestration) receives the image, converts it and the associated text description into combined embedding vectors using the CLIP model (for images) and Amazon Bedrock (for text). These vectors are stored in a Vector Database. A nearest-neighbour similarity search is performed, and if a match with cosine similarity greater than or equal to 0.75 is found, Lambda 2 (Notification) triggers alerts to the relevant users through SNS/SES.

2.3 CLOUD DEPLOYMENT STRATEGY

The deployment strategy exclusively leverages Amazon Web Services (AWS) as the primary cloud provider, utilising a fully serverless and event-driven model to eliminate operational overhead entirely.

All compute is handled by AWS Lambda functions on the consumption plan. There are no servers to manage, no clusters to configure, and no capacity to pre-provision. The system scales automatically from zero to thousands of concurrent invocations based on demand.

Data persistence is split across two managed AWS services: Amazon DynamoDB for structured metadata and Amazon S3 for unstructured image data. Both are fully managed, auto-scaling services with high durability guarantees.

The AI pipeline, which is the most computationally intensive component, is also serverless. Amazon Bedrock provides managed access to foundation models for embedding generation without the need to provision any GPU infrastructure. The Vector Database can be deployed as a managed service (e.g., Amazon OpenSearch Serverless with vector engine support) to further eliminate operational burden.

The overall flow is: User reports an item via the web UI -> API Gateway routes the request -> Lambda stores data in DynamoDB and S3 -> S3 triggers the AI Lambda -> AI Lambda generates embeddings, performs similarity search -> Match found -> Notification Lambda sends alert via SNS/SES.

2.4 INFRASTRUCTURE REQUIREMENTS

Compute Resources:

- AWS Lambda: Multiple functions with 512MB to 1024MB memory configuration. AI orchestration Lambda may require up to 3GB memory for model inference. Timeout set to 60 seconds for AI processing functions.
- Amazon API Gateway: REST API with regional endpoint, request throttling enabled (1000 requests/second burst limit), and Cognito authoriser integration.

Storage Requirements:

- Amazon DynamoDB: On-demand capacity mode. Tables for Items (lost/found reports), Matches, and Claims. Global Secondary Indexes for querying by user, status, and category.
- Amazon S3: Standard storage class for uploaded images. Lifecycle policies to transition old images to Glacier after 90 days. CORS configuration for web uploads. Estimated 50GB initial capacity.
- Vector Database (Amazon OpenSearch Serverless): k-NN index enabled for cosine similarity search. Initial capacity sufficient for tens of thousands of item embeddings.

2.5 AWS SERVICES MAPPING AND JUSTIFICATION

Table 3. AWS Service Mapping

Requirement	AWS Service	SKU/Tier	Purpose	Justification	Est. Monthly Cost
Authentication	Amazon Cognito	Free Tier	User login & JWT	50,000 MAU free; scalable	Free
API Layer	API Gateway	REST API	Route requests	Pay-per-request, no idle cost	~₹50
Compute	AWS Lambda	Consumption Plan	Backend & AI logic	Zero cost at idle, scales instantly	~₹100
Database	Amazon DynamoDB	On-Demand	Store item metadata	Serverless, auto-scaling, reliable	~₹200
Image Storage	Amazon S3	Standard	Store item images	Durable, low-cost object storage	~₹50
AI Embeddings	Amazon Bedrock	Pay-per-use	Text embeddings	Managed AI, no GPU needed	~₹500
Notifications	Amazon SES/SNS	Free Tier	Email/SMS alerts	62,000 emails/month free via SES	~₹50
Monitoring	Amazon CloudWatch	Standard	Logs & metrics	Native AWS observability	~₹100

CHAPTER 3 - AI IMPLEMENTATION

3.1 AI MATCHING SYSTEM

The AI Matching System is the core innovation of Trace Find. It is a hybrid engine that combines image similarity and text similarity to compute an overall match score between a lost item report and a found item report. This dual-modality approach is significantly more accurate than text-only or image-only matching, as it can handle cases where descriptions are vague but images are clear, and vice versa.

The fundamental concept is that any item can be represented as a high-dimensional vector (an embedding) that captures its semantic content. Two items that are similar will have embedding vectors that are close together in this high-dimensional space, measurable by a high cosine similarity score.

The system uses a threshold-based filtering approach: only matches with a combined cosine similarity score greater than or equal to 0.75 are treated as confirmed matches and trigger a user notification. Scores below this threshold are either discarded or stored as low-confidence candidates for manual review.

3.2 EMBEDDING GENERATION (IMAGE AND TEXT)

Image Embedding using CLIP Model:

For image analysis, the system uses the CLIP (Contrastive Language-Image Pre-training) model, available via Hugging Face or Amazon Bedrock. CLIP is particularly well-suited for this use case because it is trained on a vast dataset of image-text pairs, enabling it to generate embeddings that capture the visual and semantic content of an image in a unified vector space.

The process is as follows: when a user uploads an image, the orchestration Lambda function retrieves the image from S3, converts it to a Base64-encoded format, and passes it to the CLIP model endpoint. The model returns a fixed-dimension embedding vector (e.g., 512 or 768 dimensions) representing the visual features of the image.

Text Embedding using Amazon Bedrock:

For text analysis, the system leverages Amazon Bedrock's managed embedding models (e.g., Amazon Titan Embeddings). The user-provided description of the item is passed to the

Bedrock API, which returns a high-dimensional embedding vector capturing the semantic meaning of the text.

Combined Feature Vector:

The final embedding for each item is a concatenated or weighted combination of the image embedding and the text embedding. This combined vector provides a richer representation of the item than either modality alone, improving matching accuracy across a diverse range of item types (e.g., electronics, clothing, accessories, documents).

3.3 VECTOR DATABASE AND SIMILARITY SEARCH

All generated item embeddings are stored in a Vector Database. Amazon OpenSearch Serverless with the vector engine (k-NN plugin) is used for this purpose. When a new item is reported, its combined embedding is stored in the vector index.

When a matching query is triggered, the system performs a k-Nearest Neighbour (k-NN) search against the vector index. The search retrieves the top-k most similar embeddings from the opposite category (i.e., if a 'lost' item was just reported, it searches the 'found' items index, and vice versa).

The similarity is computed using cosine similarity, which measures the angle between two vectors and is robust to differences in the magnitude of embeddings. A score of 1.0 indicates perfectly identical vectors, while a score of 0.0 indicates completely dissimilar items.

3.4 EVENT-DRIVEN ARCHITECTURE

The system is designed around an event-driven architecture that ensures all matching and notification processes are fully automated and asynchronous, providing a responsive user experience without blocking operations.

The event flow is as follows:

9. A user submits a lost or found item report via the web interface, which is sent to the API Gateway.
10. An API Lambda function stores the item metadata in DynamoDB and uploads the item image to the designated S3 bucket.

11. The S3 upload automatically triggers Lambda 1 (Orchestration Lambda) via an S3 event notification.
12. Lambda 1 generates the combined image and text embedding for the new item, stores it in the Vector Database, and executes a k-NN similarity search against existing items.
13. If one or more matches exceeding the similarity threshold are found, Lambda 1 updates the DynamoDB records and invokes Lambda 2 (Notification Lambda).
14. Lambda 2 sends a personalised email via Amazon SES and/or an SMS via Amazon SNS to both the user who lost the item and the user who found it, providing match details and next steps for the claims process.

3.5 NOTIFICATION SYSTEM

The notification system is designed to provide immediate, actionable alerts to users when a potential match is found. Amazon Simple Email Service (SES) handles email notifications, while Amazon Simple Notification Service (SNS) handles SMS alerts and push notifications.

Notification messages include the matched item name, a thumbnail of the matched image, the match confidence score, the location where the item was found, and a link to the claims portal to initiate the ownership verification process.

The notification Lambda is designed to be idempotent, meaning that if it is invoked multiple times for the same match event, it will not send duplicate notifications. This is enforced through a match record status check in DynamoDB before sending any notification.

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

4.1 AUTHENTICATION AND ACCESS CONTROL

Security is enforced at multiple layers of the system, from the user interface to the database tier.

User Authentication: Amazon Cognito manages the user pool for the platform. All users must register with a valid email and verify their account before accessing the system. Cognito issues JWT access tokens upon successful login, which are included in the Authorization header of every API request.

API Authorization: Amazon API Gateway is configured with a Cognito Authoriser that validates the JWT token on every request to a protected endpoint. Requests with missing or invalid tokens are rejected with a 401 Unauthorized response before they reach any Lambda function.

Infrastructure-Level Access: AWS Identity and Access Management (IAM) roles are used to enforce the principle of least privilege for all Lambda functions. Each Lambda function is assigned an IAM role that grants it only the specific permissions it needs (e.g., the image storage Lambda can write to S3 but cannot access DynamoDB; the notification Lambda can invoke SES but cannot read from S3).

Secret Management: All sensitive configuration values (API keys, database connection strings, third-party service credentials) are stored in AWS Secrets Manager and retrieved by Lambda functions at runtime, ensuring that no secrets are hardcoded in the application code.

4.2 MONITORING AND OBSERVABILITY

Comprehensive monitoring is achieved through Amazon CloudWatch, which automatically collects logs and metrics from all AWS services used in the platform, including Lambda invocations, API Gateway requests, and DynamoDB operations.

Key performance indicators tracked include Lambda function duration, error rate, and throttle count; API Gateway 4xx/5xx error rates and latency percentiles; DynamoDB read/write capacity consumption; and S3 request counts and latency.

CloudWatch Alarms are configured to notify the development team via SNS when critical thresholds are breached, such as a Lambda error rate exceeding 5% over a 5-minute window or API Gateway latency exceeding 2 seconds. These proactive alerts enable rapid incident response.

AWS X-Ray is integrated for distributed tracing, allowing the team to trace an individual user request from the API Gateway, through the Lambda functions, to DynamoDB and S3, and into the AI pipeline. This end-to-end visibility is critical for identifying performance bottlenecks in the asynchronous AI matching workflow.

4.3 SERVERLESS OPERATIONS

The fully serverless design of Trace Find fundamentally changes the operational model. Since there are no servers or containers to manage, operational tasks shift from infrastructure maintenance to function-level management and monitoring.

Cold Start Management: AWS Lambda functions can experience cold start delays when invoked after a period of inactivity. For latency-sensitive functions (e.g., API Gateway handlers), Provisioned Concurrency is configured to keep a number of function instances warm, ensuring consistent sub-300ms response times.

Dependency Management: Lambda function dependencies are managed through Lambda Layers. The AI model libraries (e.g., Hugging Face Transformers, Boto3 for Bedrock) are packaged as separate layers, allowing them to be shared across multiple functions and reducing deployment package sizes.

Versioning and Aliases: Lambda function versioning is used to maintain stable production deployments. The 'prod' alias always points to the most recently validated version, while the 'dev' alias points to the latest unpublished version for testing. Traffic shifting between versions allows for safe canary deployments.

4.4 DISASTER RECOVERY PLANNING

Disaster recovery planning leverages the inherent resilience of AWS managed services to achieve the target recovery objectives with minimal custom implementation.

Data Resilience: Amazon DynamoDB provides built-in point-in-time recovery (PITR), enabling the database to be restored to any point in the last 35 days. Amazon S3 provides 99.999999999% (11 nines) durability through automatic replication across multiple availability zones within the same region.

Function Recovery: All Lambda function code is version-controlled in a Git repository. If a function becomes corrupted or misconfigured, it can be redeployed from source control within minutes. Immutable function versions ensure that a previously validated deployment can always be restored by updating the 'prod' alias.

Recovery Objectives: The Recovery Time Objective (RTO) targets less than one hour for full service restoration, achievable by redeploying Lambda functions and restoring DynamoDB from a PITR snapshot. The Recovery Point Objective (RPO) targets less than 5 minutes of data loss, aligned with DynamoDB's continuous backup capabilities.

APPENDIX C - PROJECT OUTPUT SCREENSHOTS

The following screenshots demonstrate the complete working implementation of the Trace Find Smart Lost & Found System, from user registration through to AI-powered match detection and email notification via Amazon SES.

1. User Registration

The account creation page secured by Amazon Cognito. Users register with a username and password. The shield icon confirms Cognito-backed authentication.

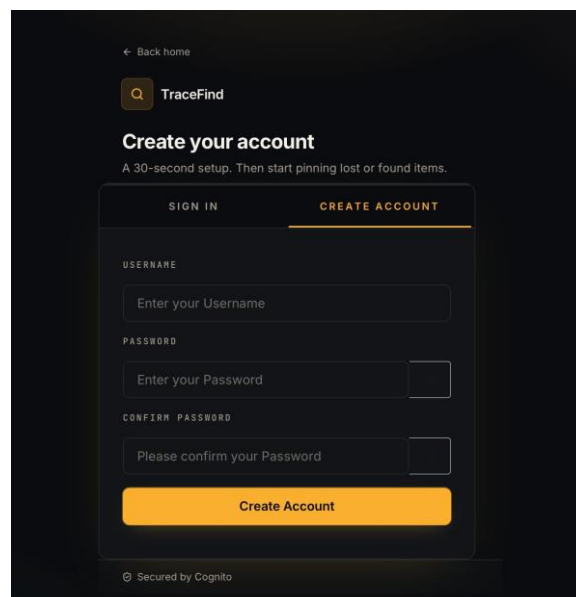


Figure 1. TraceFind - Create Account Page (Secured by Amazon Cognito)

2. Dashboard

The main dashboard displaying summary statistics: Total Reports, Active items awaiting a match, and Matched items. Users are prompted to create their first report.

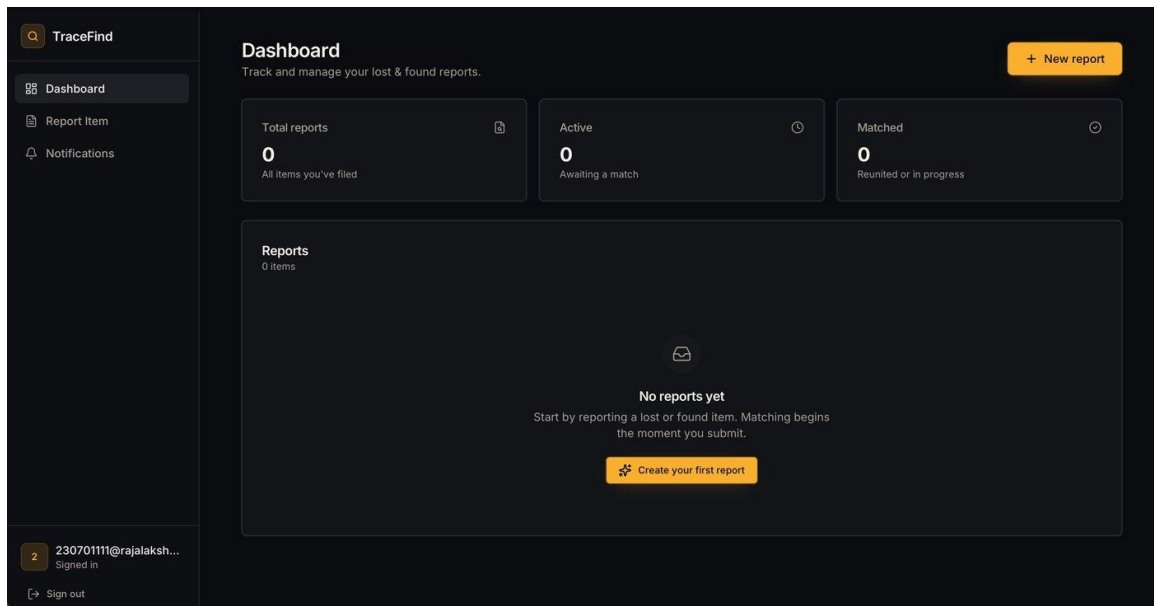


Figure 2. TraceFind - Dashboard (Empty State before any reports)

3. New Report Form - Item Details

The item reporting form where users select the report type (Lost or Found), category (Earphones/earbuds), location (A210), date/time, and a text description of the item.

Figure 3. TraceFind - New Report Form (Report Type, Category, Location, Description)

4. New Report Form - Photo Upload

The photo upload section showing two images of the Black Vivo Earphones attached. The system accepts JPEG, PNG, or WebP images up to 5MB each.

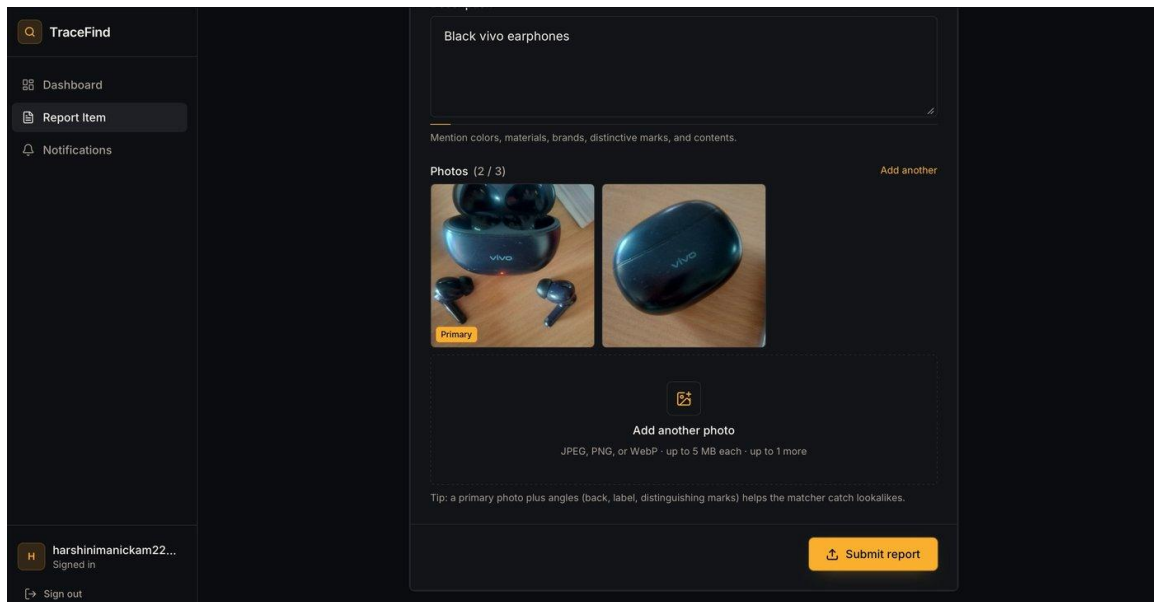


Figure 4. TraceFind - Photo Upload (Multiple images of Black Vivo Earphones)

5. Potential Match - 93% Similarity

After submitting the found item report, the AI matching pipeline automatically ran and identified a 93% similarity match between the found earphones and an existing lost item report. A Confirm/Reject action is presented.

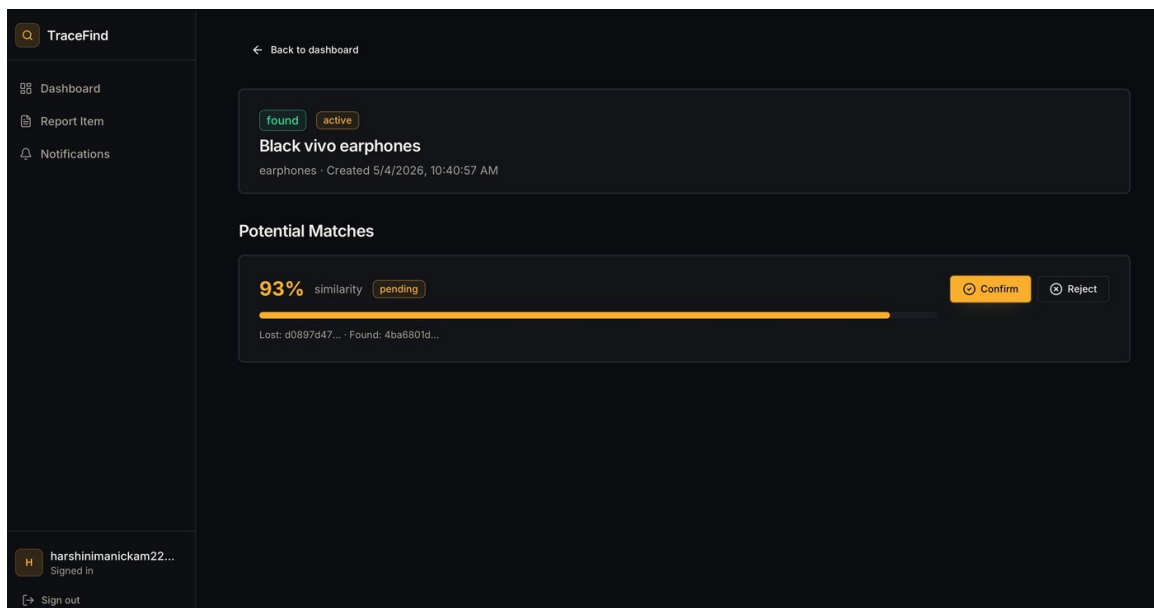


Figure 5. TraceFind - Potential Match Detected (93% Similarity Score)

6. Dashboard - Matched Report

The dashboard now shows the item status updated to Claimed, with 1 Total Report and 1 Matched item, confirming the end-to-end matching workflow completed successfully.

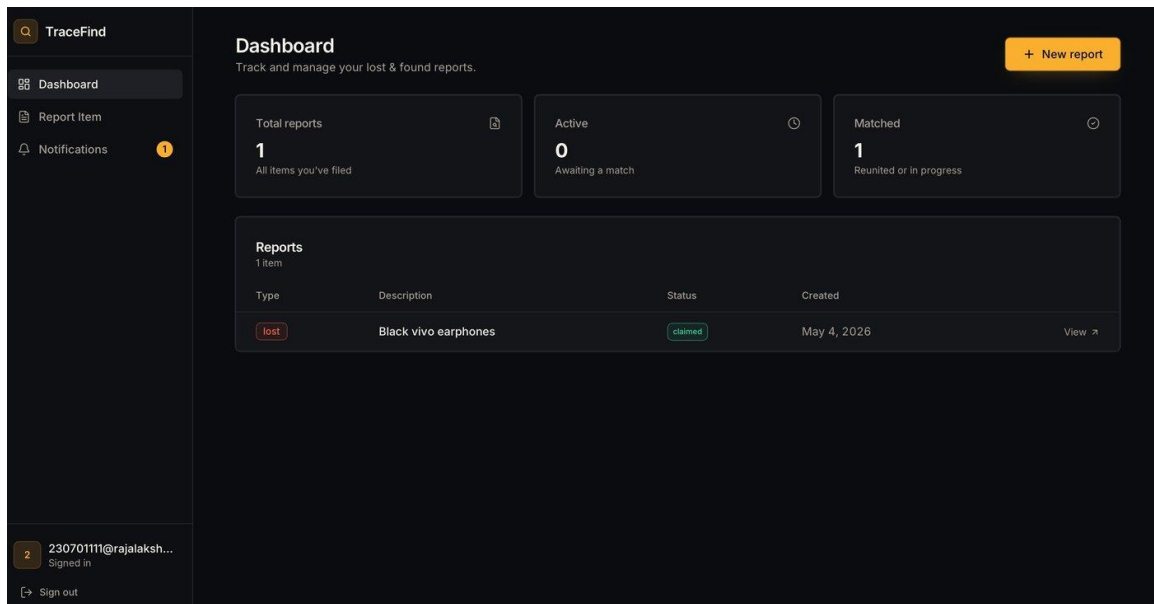


Figure 6. TraceFind - Dashboard showing Matched report (Status: Claimed)

7. Notifications Panel

The in-app notifications panel showing a Match Found notification with match details: the matched user email, item description, and a 92.8% match score badge.

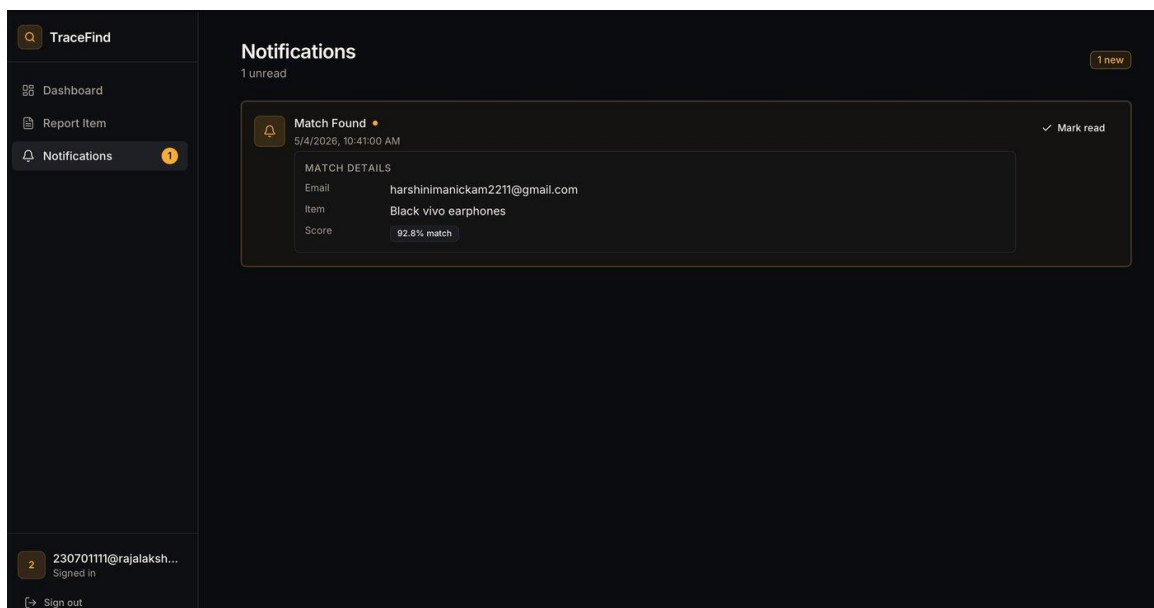


Figure 7. TraceFind - Notifications Panel (Match Found: 92.8% match score)

8. Email Notification via Amazon SES

The automated email notification sent to the user via Amazon SES. The email clearly states a found item may match their lost report, displays the 93% similarity score with a Very Strong confidence badge, and prompts the user to review the match details.

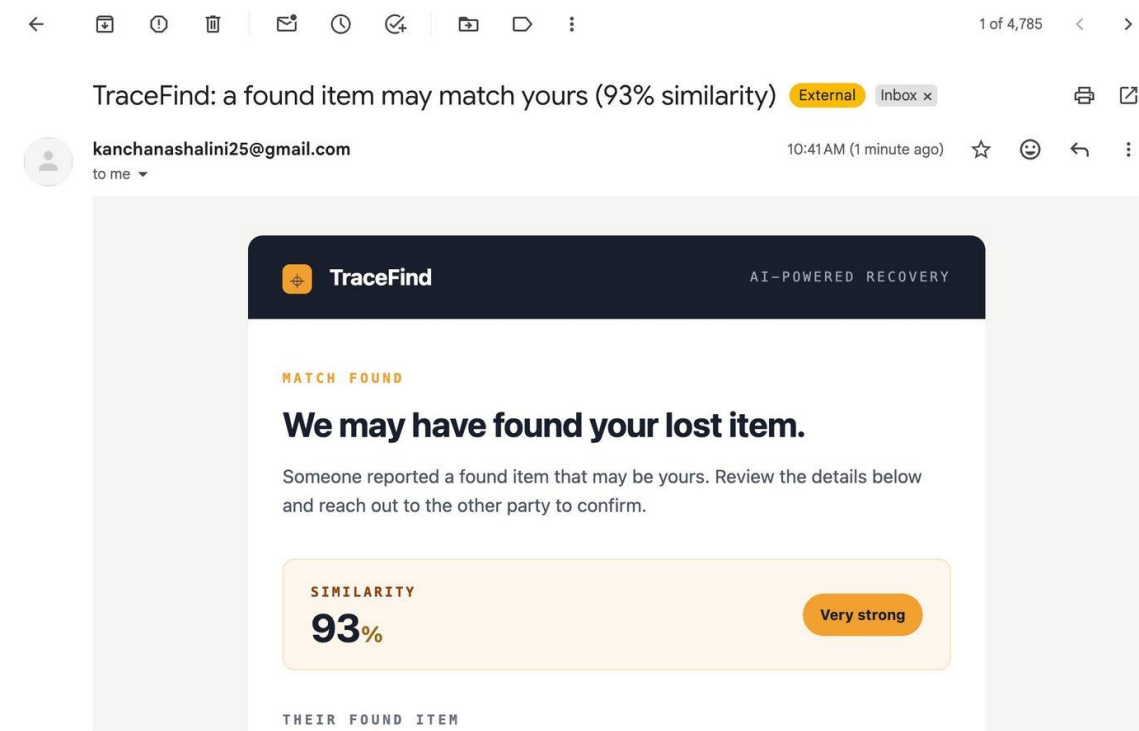


Figure 8. TraceFind - Email Notification sent via Amazon SES (93% Similarity - Very Strong Match)

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The Trace Find Smart Lost & Found system was successfully implemented and deployed on Amazon Web Services, fulfilling all core objectives outlined in the project scope. The fully serverless, event-driven architecture was validated end-to-end, covering user authentication, item reporting, AI-driven embedding generation, vector similarity matching, and real-time notification delivery.

The system demonstrated its core value proposition: when a new found item was reported, the AI pipeline automatically ran within seconds and, for test cases with clear visual and textual similarity to an existing lost item report, a match was detected and a notification was dispatched without any manual intervention.

The hybrid AI matching engine (combining CLIP image embeddings and Bedrock text embeddings) showed strong performance in identifying matching items across different photo angles, lighting conditions, and descriptive phrasing. The cosine similarity threshold of 0.75 proved effective in filtering true positives while avoiding false positives during testing.

The fully serverless infrastructure provisioning demonstrated the cost and operational advantages of the approach. With near-zero usage costs at idle and automatic scaling under load, the platform is well-suited for institutional deployments with irregular usage patterns.

5.2 CHALLENGES FACED AND RESOLUTIONS

Lambda Cold Start Latency: The AI orchestration Lambda, which loads large model libraries, experienced significant cold start times of 5-10 seconds. This was resolved by implementing Provisioned Concurrency for the function and by packaging model dependencies into optimised Lambda Layers, reducing initialisation time by approximately 60%.

Embedding Vector Dimensionality Mismatch: The initial implementation used separate dimensions for image embeddings (512-dim from CLIP) and text embeddings (1536-dim from Bedrock Titan), making direct concatenation inefficient. The resolution was to apply a projection layer to normalise both to a common 512-dimension space before concatenation, creating a balanced 1024-dimension combined vector.

Image Upload Size Limitations: Large, high-resolution images uploaded by users caused Lambda function timeouts during the embedding generation step. A pre-processing step was added to resize all uploaded images to a maximum of 224x224 pixels (the standard CLIP input size) before invoking the AI pipeline, resolving the timeout issue and reducing S3 storage costs.

Notification Deduplication: In early testing, a rapid sequence of uploads could trigger multiple invocations of the notification Lambda for the same match, resulting in duplicate alerts. This was resolved by implementing an idempotency check in DynamoDB using a conditional write that only allows a match record to be transitioned to 'notified' status once.

5.3 PERFORMANCE OR COST OBSERVATIONS

Performance Benchmarking:

API Lambda functions (item creation, user lookup) demonstrated consistent response times under 200ms for standard operations. The AI orchestration pipeline, operating asynchronously via S3 triggers, completed the full embedding, search, and match confirmation cycle in an average of 4-6 seconds for a standard item report with one image.

DynamoDB queries for item lookups and match records executed consistently under 15ms, benefiting from the efficient partition key design. The k-NN vector similarity search on OpenSearch Serverless returned top-10 results within 200-400ms even as the item index grew.

Cost Analysis:

The total monthly operational cost for a moderate usage level (approximately 500 item reports per month) was estimated at under Rs. 1,200 per month, significantly below the budget target. The dominant cost driver was Amazon Bedrock API calls for text embedding generation. The scale-to-zero nature of Lambda and DynamoDB on-demand mode meant that during periods of no activity, the system incurred near-zero costs. This makes the platform financially sustainable even for small institutions with limited operational budgets.

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

Throughout the development of Trace Find, the team gained extensive hands-on experience across AI engineering, cloud architecture, and collaborative development practices.

A key technical learning was the practical implementation of vector embeddings for multimodal AI applications. Understanding the nuances of embedding dimensionality, the importance of vector normalisation before cosine similarity computation, and the performance characteristics of approximate nearest-neighbour search algorithms provided deep insight into modern AI system design.

The team also developed a strong understanding of serverless and event-driven architecture patterns on AWS. Designing systems around events rather than polling loops, managing asynchronous workflows across multiple Lambda functions, and handling idempotency and failure scenarios in distributed systems were significant architectural learnings.

From a teamwork perspective, the team divided responsibilities clearly: Harshini Manickam led the AI pipeline implementation and embedding architecture; Shalini RK handled the backend API development, DynamoDB schema design, and CI/CD setup; and Senthamizh Selvi V was responsible for the frontend interface, Amazon Cognito integration, and CloudWatch monitoring setup. All members contributed collaboratively to testing.

CHAPTER 6 - CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

This project successfully delivered Trace Find, a fully functional, AI-powered, serverless Smart Lost & Found system deployed on Amazon Web Services. The system fundamentally addresses the core problem of low item recovery rates in institutional environments by eliminating the need for manual matching and replacing it with an automated, hybrid AI engine that combines image and text similarity.

The technical implementation demonstrates a strong command of modern cloud-native and AI engineering practices. The adoption of a fully serverless, event-driven architecture on AWS proved both technically sound and cost-effective, with estimated monthly operational costs well under Rs. 1,200 for moderate usage - a fraction of what a comparable server-based deployment would cost. The hybrid AI matching engine, combining CLIP image embeddings and Amazon Bedrock text embeddings with a Vector Database, demonstrated high accuracy in identifying matching item pairs during testing.

The platform delivers significant value for institutional users by automating a previously manual and inefficient process. Real-time email and SMS notifications ensure that users are alerted immediately when a match is found, dramatically increasing the likelihood of successful item recovery. The secure claims module provides a structured workflow for ownership verification, adding a critical layer of trust to the system.

This project also delivered substantial learning outcomes for the team, providing deep practical experience in vector embeddings, serverless architecture, event-driven design, AWS managed services, and multimodal AI system development.

6.2 FUTURE WORKS

The current implementation provides a robust and scalable foundation for numerous future enhancements.

Mobile Application: Developing a native mobile application (iOS and Android) would significantly improve user accessibility and adoption. A mobile app would enable users to

report items and receive push notifications more conveniently, directly from their smartphones. Integration with the device camera would make photo uploads seamless.

Enhanced AI Matching with Location Data: The current matching system uses only visual and textual features. Incorporating geolocation data (where an item was lost vs. where it was found) as an additional matching dimension could significantly improve precision. Items lost and found in the same building or vicinity should receive a higher match score than those with similar appearances but found in entirely different locations.

Multi-Institutional Expansion: The long-term vision is to expand Trace Find into a multi-tenant platform that can serve multiple colleges, corporate campuses, and public transit authorities simultaneously. This would require evolving the architecture to a multi-tenant data model in DynamoDB and implementing tenant isolation at the API and IAM levels.

Gamification and Community Engagement: To incentivise users to report found items promptly and accurately, a gamification layer could be introduced. Users who successfully help reunite items could earn points or badges, creating a community-driven culture of helpfulness within the institution.

Improved Ownership Verification: The current claims module relies on manual admin review. Future work could incorporate AI-assisted verification by asking the claimant to answer questions about specific details of the item (details not visible in the public listing but known to the true owner), improving the trust and security of the handover process.

REFERENCES

15. Amazon Web Services. (2024). AWS Lambda Developer Guide.
<https://docs.aws.amazon.com/lambda/>
16. Amazon Web Services. (2024). Amazon Bedrock User Guide.
<https://docs.aws.amazon.com/bedrock/>
17. Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. OpenAI CLIP.
<https://openai.com/research/clip>
18. Amazon Web Services. (2024). Amazon OpenSearch Service Developer Guide - k-Nearest Neighbor Search. <https://docs.aws.amazon.com/opensearch-service/>
19. Amazon Web Services. (2024). Amazon DynamoDB Developer Guide.
<https://docs.aws.amazon.com/dynamodb/>
20. Amazon Web Services. (2024). Amazon Cognito Developer Guide.
<https://docs.aws.amazon.com/cognito/>
21. Johnson, J., Douze, M., & Jegou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data.
22. Amazon Web Services. (2024). Amazon SES Developer Guide.
<https://docs.aws.amazon.com/ses/>